Практическая работа №7. Пользовательские функции в MS SQL Server

Цель работы

Научиться создавать и использовать пользовательские функции в MS SQL Server:

- создавать scalar-функции;
- передавать параметры в функции;
- возвращать вычисленное значение;
- создавать inline table-valued functions;
- использовать функции в `SELECT`, `WHERE` и `FROM`;
- понимать различия между функцией, представлением и хранимой процедурой.

Сценарий

Представим, что `CollegeDB` используется как небольшая информационная система колледжа.

В базе уже есть:

- `college.groups` — учебные группы;
- `college.students` — студенты;
- `college.grades` — оценки студентов.

Вам необходимо добавить несколько функций, которые упростят получение информации о студентах.

Подготовка

Используйте базу данных `CollegeDB`.

Проверьте наличие таблиц:

```
USE CollegeDB;

SELECT * FROM college.groups;
SELECT * FROM college.students;
SELECT * FROM college.grades;
```

Задание 1. Scalar-функция для определения статуса студента

Создайте функцию:

```
college.GetStudentStatus
```

Функция должна принимать средний балл студента и возвращать текстовый статус.

Правила:

Средний балл	Статус
4.5–5.0	Отличник
3.5–4.49	Хорошист
ниже 3.5	Требуется дополнительная подготовка

Пример использования:

```
SELECT
    name,
    average_grade,
```

```
    college.GetStudentStatus(average_grade) AS status
FROM college.students;
```

Проверьте функцию отдельно:

```
SELECT college.GetStudentStatus(5.0);
SELECT college.GetStudentStatus(4.2);
SELECT college.GetStudentStatus(3.1);
```

Задание 2. Функция для расчёта скидки

Создайте scalar-функцию:

```
college.CalculateDiscount
```

Функция должна принимать:

- исходную стоимость;
- процент скидки.

Например:

```
10000 ₽ + скидка 10% = 9000 ₽
```

Функция должна возвращать итоговую стоимость.

Проверьте её:

```
SELECT college.CalculateDiscount(10000, 10);
SELECT college.CalculateDiscount(25000, 15);
```

Дополнительно проверьте функцию непосредственно в запросе:

```
SELECT
    name,
```

```
    average_grade,
    college.CalculateDiscount(10000, 10) AS discounted_price
FROM college.students;
```

Задание 3. Использование функции в аналитическом запросе

Получите список студентов:

- имя;
- название группы;
- средний балл;
- статус студента.

Статус необходимо получать именно через созданную функцию.

Результат должен выглядеть примерно так:

```
Имя | Группа | Средний балл | Статус
_____________________________________
```

Для получения названия группы используйте `JOIN`.

Задание 4. Inline Table-Valued Function

Создайте табличную функцию:

```
college.GetStudentsByGroup
```

Функция должна принимать `@group_id` и возвращать студентов указанной группы.

В результат включите:

- ID студента;

- имя;
- возраст;
- средний балл;
- ID группы.

Пример использования:

```
SELECT *
FROM college.GetStudentsByGroup(1);
```

Проверьте функцию для нескольких групп.

Задание 5. Использование TVF в JOIN

Используйте созданную функцию вместе с таблицей `college.groups`.

Получите:

- имя студента;
- название группы;
- средний балл.

Пример структуры запроса:

```
SELECT
    s.name,
    g.name AS group_name,
    s.average_grade
FROM college.GetStudentsByGroup(1) s
JOIN college.groups g
    ON g.id = s.group_id;
```

Задание 6. Функция для определения возраста

Создайте scalar-функцию:

```
college.GetAgeCategory
```

Функция получает возраст и возвращает категорию:

- до 18 лет — `Несовершеннолетний`;
- 18–20 лет — `Студент`;
- старше 20 лет — `Взрослый`.

Проверьте её на нескольких значениях:

```sql
SELECT college.GetAgeCategory(17);
SELECT college.GetAgeCategory(19);
SELECT college.GetAgeCategory(25);
```

Затем примените функцию ко всем студентам:

```sql
SELECT
    name,
    age,
    college.GetAgeCategory(age) AS age_category
FROM college.students;
```

Итоговое задание

Создайте итоговый аналитический запрос, который выводит:

- имя студента;
- название группы;
- возраст;
- средний балл;
- статус успеваемости;

- возрастную категорию.

При этом:

- статус должен определяться через `GetStudentStatus`;
- возрастная категория — через `GetAgeCategory`;
- название группы необходимо получить через `JOIN`.

Пример структуры результата:

```
Алексей | ИС-31 | 18 | 4.7 | Отличник | Студент
Мария   | ПР-31 | 19 | 4.1 | Хорошист | Студент
...
```

Требования к сдаче

Один файл:

```
Практика_Функции_ФИО.sql
```

Файл должен содержать:

1. создание всех функций;
2. тестирование каждой функции;
3. запросы с использованием функций;
4. итоговый аналитический запрос.

Все созданные функции должны находиться в схеме `college`.

Вопросы для самопроверки

1. Что такое пользовательская функция?
2. Чем scalar-функция отличается от table-valued function?
3. Что возвращает scalar-функция?
4. Что возвращает inline table-valued function?

5. Где можно использовать пользовательскую функцию?
6. Чем функция отличается от хранимой процедуры?
7. Чем функция отличается от `VIEW`?
8. Зачем функции передаются параметры?
9. Почему функция должна возвращать определённый тип данных?
10. В каких ситуациях использование функции может упростить SQL-запрос?

Что должно быть создано

В результате работы в базе должны существовать:

```
college.GetStudentStatus
college.CalculateDiscount
college.GetAgeCategory
college.GetStudentsByGroup
```

Итог

В ходе работы вы создали собственные функции SQL Server и использовали их как часть обычных SQL-запросов.